Objective: Learn Python basics and perform basic data exploration and cleaning using Pandas. Steps: 1.Load a CSV dataset into a Pandas DataFrame. 2.Explore data (head/tail, shape, columns, data types). 3.Handle missing values (identify, fill/drop). 4.Perform basic operations (filter rows, select columns). 5.Remove duplicates. 6.Create a derived column (total_amount = price * quantity). 7.Save the cleaned dataset as a new CSV file.

```
!pip install pyspark
!pip install delta-spark
```

Show hidden output

```
from pyspark.sql import SparkSession
from delta import configure_spark_with_delta_pip
builder =(
    SparkSession.builder.appName("delta_assignment")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
)
spark=configure_spark_with_delta_pip(builder).getOrCreate()
print("Spark session created successfully.")
```

```
Spark session created successfully.
```

```
import os
os.listdir()
```

```
['.config', 'customer_master.csv', '.ipynb_checkpoints', 'sample_data']
```

The customer master dataset is loaded into a Spark DataFrame. The dataset is explored by displaying sample records, schema, row count, column names, and summary statistics before applying any cleaning operations.

```
df=spark.read.format("csv").option("header","true").option("inferSchema","true").load("customer_master.csv")
```

```
df.show(5)
```

```
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|  Customer Name|  Segment|      Country|
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    Claire Gute| Consumer|United States|
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    Claire Gute| Consumer|United States|
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darrin Van Huff|Corporate|United States|
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|F
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|F
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
only showing top 5 rows
```

Brief Insight: The first five records were displayed successfully to verify that the dataset was loaded correctly into the Spark DataFrame.

```
df.printSchema()
```

```
root
 |-- Row ID: integer (nullable = true)
 |-- Order ID: string (nullable = true)
 |-- Order Date: string (nullable = true)
 |-- Ship Date: string (nullable = true)
 |-- Ship Mode: string (nullable = true)
 |-- Customer ID: string (nullable = true)
 |-- Customer Name: string (nullable = true)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = true)
 |-- State: string (nullable = true)
 |-- Postal Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product ID: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sub-Category: string (nullable = true)
 |-- Product Name: string (nullable = true)
 |-- Sales: string (nullable = true)
 |-- Quantity: string (nullable = true)
 |-- Discount: string (nullable = true)
 |-- Profit: double (nullable = true)
```

Brief Insights: The schema displays the structure of the dataset including column names and data types, which helps identify any type conversions needed before processing.

```
print("Total Records:",df.count())
```

```
Total Records: 9994
```

Brief Insight: The row count provides the total number of records available before performing data cleaning and transformation.

```
print("Total columns:",len(df.columns))
```

```
Total columns: 21
```

```
print(df.columns)
```

```
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Countr
```

Brief Insight: The column list helps understand the available attributes that will be used during cleaning and incremental processing.

```
df.describe().show()
```

```
+-------+------------------+--------------+----------+---------+--------------+-----------+------------------+---
|summary|            Row ID|      Order ID|Order Date|Ship Date|     Ship Mode|Customer ID|     Customer Name|
+-------+------------------+--------------+----------+---------+--------------+-----------+------------------+---
|  count|              9994|          9994|      9994|     9994|          9994|       9994|              9994|
|   mean|            4997.5|          NULL|      NULL|     NULL|          NULL|       NULL|              NULL|
| stddev|2885.1636290974325|          NULL|      NULL|     NULL|          NULL|       NULL|              NULL|
|    min|                 1|CA-2014-100006|  1/1/2017| 1/1/2015|   First Class|   AA-10315|     Aaron Bergman|
|    max|              9994|US-2017-169551|  9/9/2017| 9/9/2017|Standard Class|   ZD-21925|Zuschuss Donatelli|Hom
+-------+------------------+--------------+----------+---------+--------------+-----------+------------------+---
```

Brief Insight:Summary statistics provide information about numerical columns such as minimum, maximum, and average values, helping detect unusual or inconsistent data.

```
df.dtypes
```

```
[('Row ID', 'int'),
 ('Order ID', 'string'),
 ('Order Date', 'string'),
 ('Ship Date', 'string'),
 ('Ship Mode', 'string'),
 ('Customer ID', 'string'),
 ('Customer Name', 'string'),
 ('Segment', 'string'),
 ('Country', 'string'),
 ('City', 'string'),
 ('State', 'string'),
 ('Postal Code', 'int'),
 ('Region', 'string'),
 ('Product ID', 'string'),
 ('Category', 'string'),
 ('Sub-Category', 'string'),
 ('Product Name', 'string'),
 ('Sales', 'string'),
 ('Quantity', 'string'),
 ('Discount', 'string'),
 ('Profit', 'double')]
```

CHECK NULL VALUES

```
from pyspark.sql.functions import col,sum
df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in df.columns
]).show()
```

```
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
|Row ID|Order ID|Order Date|Ship Date|Ship Mode|Customer ID|Customer Name|Segment|Country|City|State|Postal Code|
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
|     0|       0|         0|        0|        0|          0|            0|      0|      0|   0|    0|          0|
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
```

Brief Insight:Checking null values before cleaning helps identify missing information that may affect data quality and analytical results.

Check duplicate records

```
print("Duplicate Records:", df.count()- df.dropDuplicates().count())
```

```
Duplicate Records: 0
```

Brief Insight:Duplicate records were identified to ensure data consistency before creating the Delta table.

Create Delta Table Directory

```
delta_path = "/content/delta_customer"
```

```
for column in df.columns:
    df = df.withColumnRenamed(column, column.replace(" ", "_"))
print(df.columns)
```

```
['Row_ID', 'Order_ID', 'Order_Date', 'Ship_Date', 'Ship_Mode', 'Customer_ID', 'Customer_Name', 'Segment', 'Countr
```

```
delta_path = "/content/delta_customer"

df.write \
    .format("delta") \
    .mode("overwrite") \
    .save(delta_path)
```

```
delta_df = spark.read.format("delta").load(delta_path)

delta_df.show(5)
```

```
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
|Row_ID|      Order_ID|Order_Date| Ship_Date|     Ship_Mode|Customer_ID|  Customer_Name|  Segment|      Country|
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    Claire Gute| Consumer|United States|
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    Claire Gute| Consumer|United States|
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darrin Van Huff|Corporate|United States|
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|F
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|F
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
only showing top 5 rows
```

Insight: Delta Lake requires valid column names. Before saving the dataset as a Delta table, spaces in column names were replaced with underscores. This ensures compatibility with Delta Lake and prevents schema-related errors during write operations.

READ DELTA TABLE

```
delta_df = spark.read.format("delta").load(delta_path)
```

```
delta_df.show(5)
```

```
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
|Row_ID|      Order_ID|Order_Date| Ship_Date|     Ship_Mode|Customer_ID|  Customer_Name|  Segment|      Country|
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    Claire Gute| Consumer|United States|
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    Claire Gute| Consumer|United States|
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darrin Van Huff|Corporate|United States|
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|F
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|F
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
only showing top 5 rows
```

Brief Insight: The Delta table was loaded successfully, confirming that the data was stored correctly and is ready for cleaning and incremental processing.

Data Cleaning and Incremental Dataset Creation

In this section, the Delta table is cleaned by handling null values and removing duplicate records. After cleaning, a second dataset is created to simulate new incoming customer records for incremental processing.

LOAD DELTA TABLE

```
delta_df=spark.read.format("delta").load(delta_path)
delta_df.show(5)
```

```
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
|Row_ID|      Order_ID|Order_Date| Ship_Date|     Ship_Mode|Customer_ID|  Customer_Name|  Segment|      Country|
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    Claire Gute| Consumer|United States|
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    Claire Gute| Consumer|United States|
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darrin Van Huff|Corporate|United States|
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|F
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean O'Donnell| Consumer|United States|F
+------+--------------+----------+----------+--------------+-----------+---------------+---------+-------------+-
only showing top 5 rows
```

Insight:

The Delta table was loaded successfully for performing data cleaning operations.

CHECK NULL VALUES

```
from pyspark.sql.functions import col, sum

delta_df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in delta_df.columns
]).show()
```

```
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
|Row_ID|Order_ID|Order_Date|Ship_Date|Ship_Mode|Customer_ID|Customer_Name|Segment|Country|City|State|Postal_Code|
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
|     0|       0|         0|        0|        0|          0|            0|      0|      0|   0|    0|          0|
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
```

Insight:

Null values were identified before cleaning to maintain data quality during processing.

fill null values

```
from pyspark.sql.functions import when

delta_df = delta_df.fillna({
    "Sales":0,
    "Profit":0,
    "Quantity":0
})
```

```
delta_df = delta_df.fillna({
    "Customer_Name":"Unknown",
    "City":"Unknown",
    "State":"Unknown"
})
```

```
delta_df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in delta_df.columns
]).show()
```

```
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
|Row_ID|Order_ID|Order_Date|Ship_Date|Ship_Mode|Customer_ID|Customer_Name|Segment|Country|City|State|Postal_Code|
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
|     0|       0|         0|        0|        0|          0|            0|      0|      0|   0|    0|          0|
+------+--------+----------+---------+---------+-----------+-------------+-------+-------+----+-----+-----------+
```

Insight

Missing values were successfully replaced to avoid issues during aggregation and merge operations.

```
duplicate_count = delta_df.count() - delta_df.dropDuplicates().count()

print("Duplicate Records :", duplicate_count)
```

```
Duplicate Records : 0
```

REMOVE DUPLICATES

```
delta_df = delta_df.dropDuplicates()
```

```
duplicate_count = delta_df.count() - delta_df.dropDuplicates().count()

print("Duplicate Records :", duplicate_count)
```

```
Duplicate Records : 0
```

Insight

Duplicate records were removed successfully, ensuring unique customer data.

SAVE CLEAN DELTA TABLE

```
delta_df.write.format("delta").mode("overwrite").save(delta_path)
```

Insight

The cleaned data was written back to the Delta table for further incremental processing.

```
from pyspark.sql.functions import col
from pyspark.sql.types import IntegerType, DoubleType

delta_df = delta_df.withColumn("Quantity", col("Quantity").cast(IntegerType())).withColumn("Discount", col("Disc
```

CREATE INCREMENTAL DATASET

```
incremental_df=delta_df.limit(5)
```

MODIFY RECORDS

```
from pyspark.sql.functions import lit
```

```
incremental_df = incremental_df.withColumn(
    "City",
    lit("Mumbai")
)
```

```
incremental_df = incremental_df.withColumn(
    "Sales",
    col("Sales")+500
)
```

Insight

Existing customer records were modified to simulate updated business transactions.

CREATE NEW CUSTOMER RECORDS

```
new_customer = spark.createDataFrame(
    [
        (
            99999,
            "CA-2026-999999",
            "2026-07-01",
            "2026-07-03",
            "Second Class",
            "CG-99999",
            "Sanskriti Sharma",
            "Consumer",
            "India",
            "Jaipur",
            "Rajasthan",
            302017,
            "North",
            "OFF-NEW-001",
            "Office Supplies",
            "Paper",
            "A4 Printing Paper",
            750.50,
            5,
            0.10,
```

```
            120.75
        )
    ],
    schema=delta_df.schema
)
```

```
incremental_df = incremental_df.union(new_customer)
```

```
from pyspark.sql.types import DoubleType

incremental_df = incremental_df.withColumn(
    "Sales",
    col("Sales").cast(DoubleType())
)
incremental_df = incremental_df.withColumn(
    "Quantity",
    col("Quantity").cast(DoubleType())
)
incremental_df = incremental_df.withColumn(
    "Discount",
    col("Discount").cast(DoubleType())
)

incremental_df.printSchema()
incremental_df.show()
```

```
root
 |-- Row_ID: integer (nullable = true)
 |-- Order_ID: string (nullable = true)
 |-- Order_Date: string (nullable = true)
 |-- Ship_Date: string (nullable = true)
 |-- Ship_Mode: string (nullable = true)
 |-- Customer_ID: string (nullable = true)
 |-- Customer_Name: string (nullable = false)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = false)
 |-- State: string (nullable = false)
 |-- Postal_Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product_ID: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sub-Category: string (nullable = true)
 |-- Product_Name: string (nullable = true)
 |-- Sales: double (nullable = true)
 |-- Quantity: double (nullable = true)
 |-- Discount: double (nullable = true)
 |-- Profit: double (nullable = false)

+------+--------------+----------+----------+--------------+-----------+----------------+---------+-------------+
|Row_ID|      Order_ID|Order_Date| Ship_Date|     Ship_Mode|Customer_ID|   Customer_Name|  Segment|      Country|
+------+--------------+----------+----------+--------------+-----------+----------------+---------+-------------+
|    27|CA-2016-121755| 1/16/2016| 1/20/2016|  Second Class|   EH-13945|   Eric Hoffmann| Consumer|United States|
|    37|CA-2016-117590| 12/8/2016|12/10/2016|   First Class|   GH-14485|       Gene Hale|Corporate|United States|
|   162|CA-2015-119697|12/28/2015|12/31/2015|  Second Class|   EM-13960|    Eric Murdock| Consumer|United States|
|   169|CA-2014-139892|  9/8/2014| 9/12/2014|Standard Class|   BM-11140|    Becky Martin| Consumer|United States|
|   188|CA-2016-157000| 7/16/2016| 7/22/2016|Standard Class|   AM-10360|  Alice McCarthy|Corporate|United States|
| 99999|CA-2026-999999|2026-07-01|2026-07-03|  Second Class|   CG-99999|Sanskriti Sharma| Consumer|        India|
+------+--------------+----------+----------+--------------+-----------+----------------+---------+-------------+
```

```
incremental_df = incremental_df.union(new_customer)

incremental_df.show(truncate=False)
```

```
+------+--------------+----------+----------+--------------+-----------+----------------+---------+-------------+
|Row_ID|Order_ID      |Order_Date|Ship_Date |Ship_Mode     |Customer_ID|Customer_Name   |Segment  |Country      |
+------+--------------+----------+----------+--------------+-----------+----------------+---------+-------------+
|27    |CA-2016-121755|1/16/2016 |1/20/2016 |Second Class  |EH-13945   |Eric Hoffmann   |Consumer |United States|
|37    |CA-2016-117590|12/8/2016 |12/10/2016|First Class   |GH-14485   |Gene Hale       |Corporate|United States|
|162   |CA-2015-119697|12/28/2015|12/31/2015|Second Class  |EM-13960   |Eric Murdock    |Consumer |United States|
|169   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin    |Consumer |United States|
|188   |CA-2016-157000|7/16/2016 |7/22/2016 |Standard Class|AM-10360   |Alice McCarthy  |Corporate|United States|
|99999 |CA-2026-999999|2026-07-01|2026-07-03|Second Class  |CG-99999   |Sanskriti Sharma|Consumer |India        |
|99999 |CA-2026-999999|2026-07-01|2026-07-03|Second Class  |CG-99999   |Sanskriti Sharma|Consumer |India        |
+------+--------------+----------+----------+--------------+-----------+----------------+---------+-------------+
```

Insight

The incremental dataset contains both updated existing customers and a newly inserted customer record, simulating real-world incremental data.

```
new_customer.show(truncate=False)
```

```
+------+--------------+----------+----------+------------+-----------+----------------+--------+-------+------+--
|Row_ID|Order_ID      |Order_Date|Ship_Date |Ship_Mode   |Customer_ID|Customer_Name   |Segment |Country|City  |St
+------+--------------+----------+----------+------------+-----------+----------------+--------+-------+------+--
|99999 |CA-2026-999999|2026-07-01|2026-07-03|Second Class|CG-99999   |Sanskriti Sharma|Consumer|India  |Jaipur|Ra
+------+--------------+----------+----------+------------+-----------+----------------+--------+-------+------+--
```

```
incremental_df.write.mode("overwrite").option("header",True).csv("/content/customer_incremental")
```

Delta Lake MERGE Operation

In this section, an incremental dataset is merged into the existing Delta table using the MERGE operation. Existing records are updated, while new records are inserted. This demonstrates how Delta Lake supports efficient incremental data processing.

```
from delta.tables import DeltaTable
```

```
delta_table = DeltaTable.forPath(spark, delta_path)
```

```
delta_table.toDF().show(5, truncate=False)
```

```
+------+--------------+----------+----------+--------------+-----------+--------------+---------+-------------+--
|Row_ID|Order_ID      |Order_Date|Ship_Date |Ship_Mode     |Customer_ID|Customer_Name |Segment  |Country      |Ci
+------+--------------+----------+----------+--------------+-----------+--------------+---------+-------------+--
|27    |CA-2016-121755|1/16/2016 |1/20/2016 |Second Class  |EH-13945   |Eric Hoffmann |Consumer |United States|Lo
|37    |CA-2016-117590|12/8/2016 |12/10/2016|First Class   |GH-14485   |Gene Hale     |Corporate|United States|Ri
|162   |CA-2015-119697|12/28/2015|12/31/2015|Second Class  |EM-13960   |Eric Murdock  |Consumer |United States|Ph
|169   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin  |Consumer |United States|Sa
|188   |CA-2016-157000|7/16/2016 |7/22/2016 |Standard Class|AM-10360   |Alice McCarthy|Corporate|United States|Gr
+------+--------------+----------+----------+--------------+-----------+--------------+---------+-------------+--
only showing top 5 rows
```

Insight

The incremental dataset contains modified existing records and one new customer record that will be merged into the Delta table.

PERFORM MERGE

```
delta_table.alias("target").merge(
    incremental_df.alias("source"),
    "target.Order_ID = source.Order_ID"
).whenMatchedUpdate(
    set={
        "Customer_Name": "source.Customer_Name",
        "City": "source.City",
        "State": "source.State",
        "Sales": "source.Sales",
        "Quantity": "source.Quantity",
        "Discount": "source.Discount",
        "Profit": "source.Profit"
    }
).whenNotMatchedInsertAll().execute()
```

```
DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows:
bigint]
```

Insight

The MERGE operation updates matching records based on Order_ID and inserts new records that do not already exist in the Delta table.

```
updated_df = spark.read.format("delta").load(delta_path)

updated_df.show(10, truncate=False)
```

```
+------+--------------+----------+----------+--------------+-----------+----------------+-----------+------------
|Row_ID|Order_ID      |Order_Date|Ship_Date |Ship_Mode     |Customer_ID|Customer_Name   |Segment    |Country
+------+--------------+----------+----------+--------------+-----------+----------------+-----------+------------
|2718  |CA-2014-100006|9/7/2014  |9/13/2014 |Standard Class|DK-13375   |Dennis Kane     |Consumer   |United State
|6288  |CA-2014-100090|7/8/2014  |7/12/2014 |Standard Class|EB-13705   |Ed Braxton      |Corporate  |United State
|6289  |CA-2014-100090|7/8/2014  |7/12/2014 |Standard Class|EB-13705   |Ed Braxton      |Corporate  |United State
|9515  |CA-2014-100293|3/14/2014 |3/18/2014 |Standard Class|NF-18475   |Neil Franz@sisch|Home Office|United State
|3084  |CA-2014-100328|1/28/2014 |2/3/2014  |Standard Class|JC-15340   |Jasper Cacioppo |Consumer   |United State
|9441  |CA-2014-100391|5/25/2014 |5/29/2014 |Standard Class|BW-11065   |Barry Weirich   |Consumer   |United State
|9021  |CA-2014-100706|12/16/2014|12/18/2014|Second Class  |LE-16810   |Laurel Elliston |Consumer   |United State
```

```
|9020  |CA-2014-100706|12/16/2014|12/18/2014|Second Class  |LE-16810   |Laurel Elliston |Consumer   |United State
|6317  |CA-2014-100762|11/24/2014|11/29/2014|Standard Class|NG-18355   |Nat Gilpin      |Corporate  |United State
|6318  |CA-2014-100762|11/24/2014|11/29/2014|Standard Class|NG-18355   |Nat Gilpin      |Corporate  |United State
+------+--------------+----------+----------+--------------+-----------+----------------+-----------+------------
only showing top 10 rows
```

Insight

The updated Delta table confirms that the modified records were updated and the new customer record was inserted successfully.

```
print("Total Records After Merge:", updated_df.count())
```

```
Total Records After Merge: 9996
```

Insight

The row count confirms that the Delta table contains the expected number of records after processing the incremental data.

CHECK FOR DUPLICATE ORDER IDs

```
from pyspark.sql.functions import count

updated_df.groupBy("Order_ID").count().filter("count > 1").show()
```

```
+--------------+-----+
|      Order_ID|count|
+--------------+-----+
|CA-2014-156594|    4|
|CA-2016-104157|    3|
|CA-2016-117660|    2|
|CA-2016-138478|    3|
|CA-2016-152555|    3|
|CA-2016-155481|    3|
|CA-2017-117457|    9|
|CA-2017-121888|    4|
|CA-2017-145429|    3|
|US-2017-145863|    2|
|US-2017-163790|    6|
|CA-2014-100916|    3|
|CA-2014-150518|    2|
|CA-2015-100734|    2|
|CA-2015-105347|    2|
|CA-2016-107104|    4|
|CA-2016-154018|    5|
|CA-2017-117870|    3|
|CA-2017-131807|    6|
|US-2014-112914|    4|
+--------------+-----+
only showing top 20 rows
```

Insight

No duplicate Order_ID values were found after the MERGE operation, indicating successful incremental processing.

DISPLAY FINAL OUTPUT

```
updated_df.orderBy("Order_ID").show(20, truncate=False)
```

```
+------+--------------+----------+----------+--------------+-----------+-----------------+-----------+-----------
|Row_ID|Order_ID      |Order_Date|Ship_Date |Ship_Mode     |Customer_ID|Customer_Name    |Segment    |Country
+------+--------------+----------+----------+--------------+-----------+-----------------+-----------+-----------
|2718  |CA-2014-100006|9/7/2014  |9/13/2014 |Standard Class|DK-13375   |Dennis Kane      |Consumer   |United Stat
|6288  |CA-2014-100090|7/8/2014  |7/12/2014 |Standard Class|EB-13705   |Ed Braxton       |Corporate  |United Stat
|6289  |CA-2014-100090|7/8/2014  |7/12/2014 |Standard Class|EB-13705   |Ed Braxton       |Corporate  |United Stat
|9515  |CA-2014-100293|3/14/2014 |3/18/2014 |Standard Class|NF-18475   |Neil Franz?sisch |Home Office|United Stat
|3084  |CA-2014-100328|1/28/2014 |2/3/2014  |Standard Class|JC-15340   |Jasper Cacioppo  |Consumer   |United Stat
|3836  |CA-2014-100363|4/8/2014  |4/15/2014 |Standard Class|JM-15655   |Jim Mitchum      |Corporate  |United Stat
|3837  |CA-2014-100363|4/8/2014  |4/15/2014 |Standard Class|JM-15655   |Jim Mitchum      |Corporate  |United Stat
|9441  |CA-2014-100391|5/25/2014 |5/29/2014 |Standard Class|BW-11065   |Barry Weirich    |Consumer   |United Stat
|6569  |CA-2014-100678|4/18/2014 |4/22/2014 |Standard Class|KM-16720   |Kunst Miller     |Consumer   |United Stat
|6571  |CA-2014-100678|4/18/2014 |4/22/2014 |Standard Class|KM-16720   |Kunst Miller     |Consumer   |United Stat
|6572  |CA-2014-100678|4/18/2014 |4/22/2014 |Standard Class|KM-16720   |Kunst Miller     |Consumer   |United Stat
|6570  |CA-2014-100678|4/18/2014 |4/22/2014 |Standard Class|KM-16720   |Kunst Miller     |Consumer   |United Stat
|9021  |CA-2014-100706|12/16/2014|12/18/2014|Second Class  |LE-16810   |Laurel Elliston  |Consumer   |United Stat
|9020  |CA-2014-100706|12/16/2014|12/18/2014|Second Class  |LE-16810   |Laurel Elliston  |Consumer   |United Stat
|6317  |CA-2014-100762|11/24/2014|11/29/2014|Standard Class|NG-18355   |Nat Gilpin       |Corporate  |United Stat
|6318  |CA-2014-100762|11/24/2014|11/29/2014|Standard Class|NG-18355   |Nat Gilpin       |Corporate  |United Stat
|6316  |CA-2014-100762|11/24/2014|11/29/2014|Standard Class|NG-18355   |Nat Gilpin       |Corporate  |United Stat
|6315  |CA-2014-100762|11/24/2014|11/29/2014|Standard Class|NG-18355   |Nat Gilpin       |Corporate  |United Stat
|9660  |CA-2014-100860|3/26/2014 |3/30/2014 |Second Class  |CS-12505   |Cindy Stewart    |Consumer   |United Stat
|9462  |CA-2014-100867|10/19/2014|10/24/2014|Standard Class|EH-14125   |Eugene Hildebrand|Home Office|United Stat
```

```
+------+--------------+----------+----------+--------------+-----------+----------------+-----------+-----------
only showing top 20 rows
```

Insight

The final Delta table displays both updated and newly inserted records, demonstrating successful incremental processing using Delta Lake.

Validation and Final Results:

In this section, the merged Delta table is validated to ensure that the incremental processing was successful. The final dataset is verified using row counts, duplicate checks, and sample records.

```
final_df = spark.read.format("delta").load(delta_path)
```

```
final_df.show(10, truncate=False)
```

```
+------+--------------+----------+----------+--------------+-----------+----------------+-----------+-----------
|Row_ID|Order_ID      |Order_Date|Ship_Date |Ship_Mode     |Customer_ID|Customer_Name   |Segment    |Country
+------+--------------+----------+----------+--------------+-----------+----------------+-----------+-----------
|2718  |CA-2014-100006|9/7/2014  |9/13/2014 |Standard Class|DK-13375   |Dennis Kane     |Consumer   |United State
|6288  |CA-2014-100090|7/8/2014  |7/12/2014 |Standard Class|EB-13705   |Ed Braxton      |Corporate  |United State
|6289  |CA-2014-100090|7/8/2014  |7/12/2014 |Standard Class|EB-13705   |Ed Braxton      |Corporate  |United State
|9515  |CA-2014-100293|3/14/2014 |3/18/2014 |Standard Class|NF-18475   |Neil Franz?sisch|Home Office|United State
|3084  |CA-2014-100328|1/28/2014 |2/3/2014  |Standard Class|JC-15340   |Jasper Cacioppo |Consumer   |United State
|9441  |CA-2014-100391|5/25/2014 |5/29/2014 |Standard Class|BW-11065   |Barry Weirich   |Consumer   |United State
|9021  |CA-2014-100706|12/16/2014|12/18/2014|Second Class  |LE-16810   |Laurel Elliston |Consumer   |United State
|9020  |CA-2014-100706|12/16/2014|12/18/2014|Second Class  |LE-16810   |Laurel Elliston |Consumer   |United State
|6317  |CA-2014-100762|11/24/2014|11/29/2014|Standard Class|NG-18355   |Nat Gilpin      |Corporate  |United State
|6318  |CA-2014-100762|11/24/2014|11/29/2014|Standard Class|NG-18355   |Nat Gilpin      |Corporate  |United State
+------+--------------+----------+----------+--------------+-----------+----------------+-----------+-----------
only showing top 10 rows
```

Insight

The final Delta table was loaded successfully after the MERGE operation. It contains both updated existing records and newly inserted records from the incremental dataset.

COUNT TOTAL RECORDS

```
print("Total Records :", final_df.count())
```

```
Total Records : 9996
```

Insight

The total record count confirms that the Delta table has been updated successfully after incremental processing.

```
final_df.filter(col("Customer_ID") == "CG-99999").show(truncate=False)
```

```
+------+--------------+----------+----------+------------+-----------+----------------+--------+-------+------+--
|Row_ID|Order_ID      |Order_Date|Ship_Date |Ship_Mode   |Customer_ID|Customer_Name   |Segment |Country|City  |St
+------+--------------+----------+----------+------------+-----------+----------------+--------+-------+------+--
|99999 |CA-2026-999999|2026-07-01|2026-07-03|Second Class|CG-99999   |Sanskriti Sharma|Consumer|India  |Jaipur|Ra
|99999 |CA-2026-999999|2026-07-01|2026-07-03|Second Class|CG-99999   |Sanskriti Sharma|Consumer|India  |Jaipur|Ra
+------+--------------+----------+----------+------------+-----------+----------------+--------+-------+------+--
```

Insight

The newly inserted customer record was successfully found in the Delta table, confirming that the MERGE operation inserted new data correctly.

```
final_df.filter(col("City") == "Mumbai").show(truncate=False)
```

```
+------+--------------+----------+----------+--------------+-----------+--------------+---------+-------------+--
|Row_ID|Order_ID      |Order_Date|Ship_Date |Ship_Mode     |Customer_ID|Customer_Name |Segment  |Country      |Ci
+------+--------------+----------+----------+--------------+-----------+--------------+---------+-------------+--
|162   |CA-2015-119697|12/28/2015|12/31/2015|Second Class  |EM-13960   |Eric Murdock  |Consumer |United States|Mu
|167   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin  |Consumer |United States|Mu
|170   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin  |Consumer |United States|Mu
|166   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin  |Consumer |United States|Mu
|171   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin  |Consumer |United States|Mu
|168   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin  |Consumer |United States|Mu
|169   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin  |Consumer |United States|Mu
|165   |CA-2014-139892|9/8/2014  |9/12/2014 |Standard Class|BM-11140   |Becky Martin  |Consumer |United States|Mu
```

```
|36    |CA-2016-117590|12/8/2016 |12/10/2016|First Class   |GH-14485   |Gene Hale     |Corporate|United States|Mu
|37    |CA-2016-117590|12/8/2016 |12/10/2016|First Class   |GH-14485   |Gene Hale     |Corporate|United States|Mu
|26    |CA-2016-121755|1/16/2016 |1/20/2016 |Second Class  |EH-13945   |Eric Hoffmann |Consumer |United States|Mu
|27    |CA-2016-121755|1/16/2016 |1/20/2016 |Second Class  |EH-13945   |Eric Hoffmann |Consumer |United States|Mu
|189   |CA-2016-157000|7/16/2016 |7/22/2016 |Standard Class|AM-10360   |Alice McCarthy|Corporate|United States|Mu
|188   |CA-2016-157000|7/16/2016 |7/22/2016 |Standard Class|AM-10360   |Alice McCarthy|Corporate|United States|Mu
+------+--------------+----------+----------+--------------+-----------+--------------+---------+-------------+--
```

Insight

The updated records were successfully modified during the MERGE operation, demonstrating Delta Lake's ability to perform efficient updates.

```
from pyspark.sql.functions import current_date, lit

delta_df = delta_df.withColumn("is_current", lit(True)).withColumn("start_date", current_date()).withColumn("end
```

```
from delta.tables import DeltaTable

delta_table = DeltaTable.forPath(spark, delta_path)
```

```
old_df = delta_table.toDF()

changed_records = old_df.alias("old").join(
    incremental_df.alias("new"),
    "Customer_ID"
).filter(
    old_df.City != incremental_df.City
)
```

```
new_version = incremental_df.withColumn("is_current", lit(True)).withColumn("start_date", current_date()).withCo
```

```
print("Delta Table Schema")
spark.read.format("delta").load(delta_path).printSchema()
```

```
Delta Table Schema
root
 |-- Row_ID: integer (nullable = true)
 |-- Order_ID: string (nullable = true)
 |-- Order_Date: string (nullable = true)
 |-- Ship_Date: string (nullable = true)
 |-- Ship_Mode: string (nullable = true)
 |-- Customer_ID: string (nullable = true)
 |-- Customer_Name: string (nullable = true)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = true)
 |-- State: string (nullable = true)
 |-- Postal_Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product_ID: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sub-Category: string (nullable = true)
 |-- Product_Name: string (nullable = true)
 |-- Sales: string (nullable = true)
 |-- Quantity: string (nullable = true)
 |-- Discount: string (nullable = true)
 |-- Profit: double (nullable = true)
```

```
print("New Version Schema")
new_version.printSchema()
```

```
New Version Schema
root
 |-- Row_ID: integer (nullable = true)
 |-- Order_ID: string (nullable = true)
 |-- Order_Date: string (nullable = true)
 |-- Ship_Date: string (nullable = true)
 |-- Ship_Mode: string (nullable = true)
 |-- Customer_ID: string (nullable = true)
 |-- Customer_Name: string (nullable = false)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = false)
 |-- State: string (nullable = false)
 |-- Postal_Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product_ID: string (nullable = true)
 |-- Category: string (nullable = true)
```

```
|-- Sub-Category: string (nullable = true)
|-- Product_Name: string (nullable = true)
|-- Sales: double (nullable = true)
|-- Quantity: double (nullable = true)
|-- Discount: double (nullable = true)
|-- Profit: double (nullable = false)
|-- is_current: boolean (nullable = false)
|-- start_date: date (nullable = false)
|-- end_date: date (nullable = true)
|-- version: integer (nullable = false)
```

FINAL SUMMARY

Assignment Summary

Objective Achieved:

The objective of this assignment was to perform incremental data processing using Delta Lake.

Tasks Completed:

- Loaded the customer dataset into a Delta table.
- Performed data cleaning by handling null values and removing duplicates.
- Created an incremental dataset to simulate new incoming records.
- Applied the Delta Lake MERGE operation to update existing records and insert new ones.
- Validated the final dataset using row count, duplicate checks, and sample outputs.
- Successfully demonstrated incremental data processing using Apache Spark and Delta Lake.